Task 9: AI Pipeline Design (Armor Task)

Objective

To enhance the RAG chatbot system by implementing a robust input-output pipeline that ensures data validation, performance monitoring, and graceful failure handling for production-level reliability.

Technologies & Libraries Used

- Python
- sentence-transformers
- scikit-learn (cosine similarity)
- logging module
- re (regular expressions)
- time module

System Enhancements Over Task 8

- Input validation layer added
- Performance logging implemented
- Graceful error handling introduced
- Production-ready pipeline design

System Workflow

1. User inputs query
2. Input Validator checks query validity
3. Query is converted into embeddings
4. Similarity search retrieves relevant chunk
5. LLM generates response
6. Latency is recorded in log file

7. Response is returned to user

Key Features Implemented

Input Validation Rules:

• Reject queries longer than 300 characters

• Reject empty queries

• Reject queries with only special characters

Performance Logging:

• retrieval_time_ms: Time taken to fetch relevant chunk

• generation_time_ms: Time taken by LLM to generate response

• Logs stored in performance.log

Graceful Failure Handling:

• Prevents system crashes

• Returns user-friendly error message instead of traceback

Test Cases Implemented

• Valid query processing

• Long query rejection (>300 chars)

• Empty input handling

• Special character input rejection

• Performance logging verification

• API failure simulation

Errors Faced

• Validation not triggering correctly

• System processing invalid inputs

- Missing latency logs
- Unhandled exceptions causing crashes

Solutions Implemented

- Moved validation to first step in pipeline
- Added strict regex checks
- Integrated logging module
- Wrapped pipeline in try-except block

Output

```
PS D:\Daily Task> .\venv\Scripts\activate
(venv) PS D:\Daily Task> cd Task_9
(venv) PS D:\Daily Task\Task_9> python task9.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|| 103/103 [00:00<00:00, 3406.05it/s]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
🔄 Loading corpus...
✂️Splitting chunks...
🧠 Creating embeddings...
Batches: 100%|| 2/2 [00:00<00:00,  3.64it/s]

💬 Ask your question (type 'exit' to quit): What is Python used for?
Batches: 100%|| 1/1 [00:00<00:00, 24.59it/s]

🤖 Response:
 Answer based on context:
9.      Python is the primary language used for data science and machine learning.


💬 Ask your question (type 'exit' to quit): How do you build a RAG system?
Batches: 100%|| 1/1 [00:00<00:00, 27.31it/s]

🤖 Response:
 Answer based on context:
10.     To build a RAG system, one must first master the art of text chunking.
```

```
💬 Ask your question (type 'exit' to quit): How do you build a RAG system?
Batches: 100%|| 1/1 [00:00<00:00, 27.31it/s]

🤖 Response:
 Answer based on context:
10.     To build a RAG system, one must first master the art of text chunking.


💬 Ask your question (type 'exit' to quit):

🤖 Response:
 Input Error: Query cannot be empty.

💬 Ask your question (type 'exit' to quit): !!!???###

🤖 Response:
 Input Error: Query contains only special characters.

💬 Ask your question (type 'exit' to quit): Describe step by step how a modern AI powered application is built using Python, APIs, cloud computing, vector databa
ses, embeddings, retrieval augmented generation, prompt engineering, and monitoring tools, including how latency is handled, errors are managed, and how scalab
ility is achieved in real world enterprise systems.

🤖 Response:
 Input Error: Query too long (Max 300 characters allowed).

💬 Ask your question (type 'exit' to quit): exit
```

Conclusion

Task 9 successfully transforms the RAG chatbot into a robust and production-grade system by adding validation, monitoring, and fault tolerance. This ensures reliability, security, and better user experience.